

Air University Aerospace And Aviation Campus Kamra

Department of Computer Science



Class Diagrams

Name: Haleema Abid, Samreen Shoukat, Hamna Saleem

ID: 245053, 245077, 245047

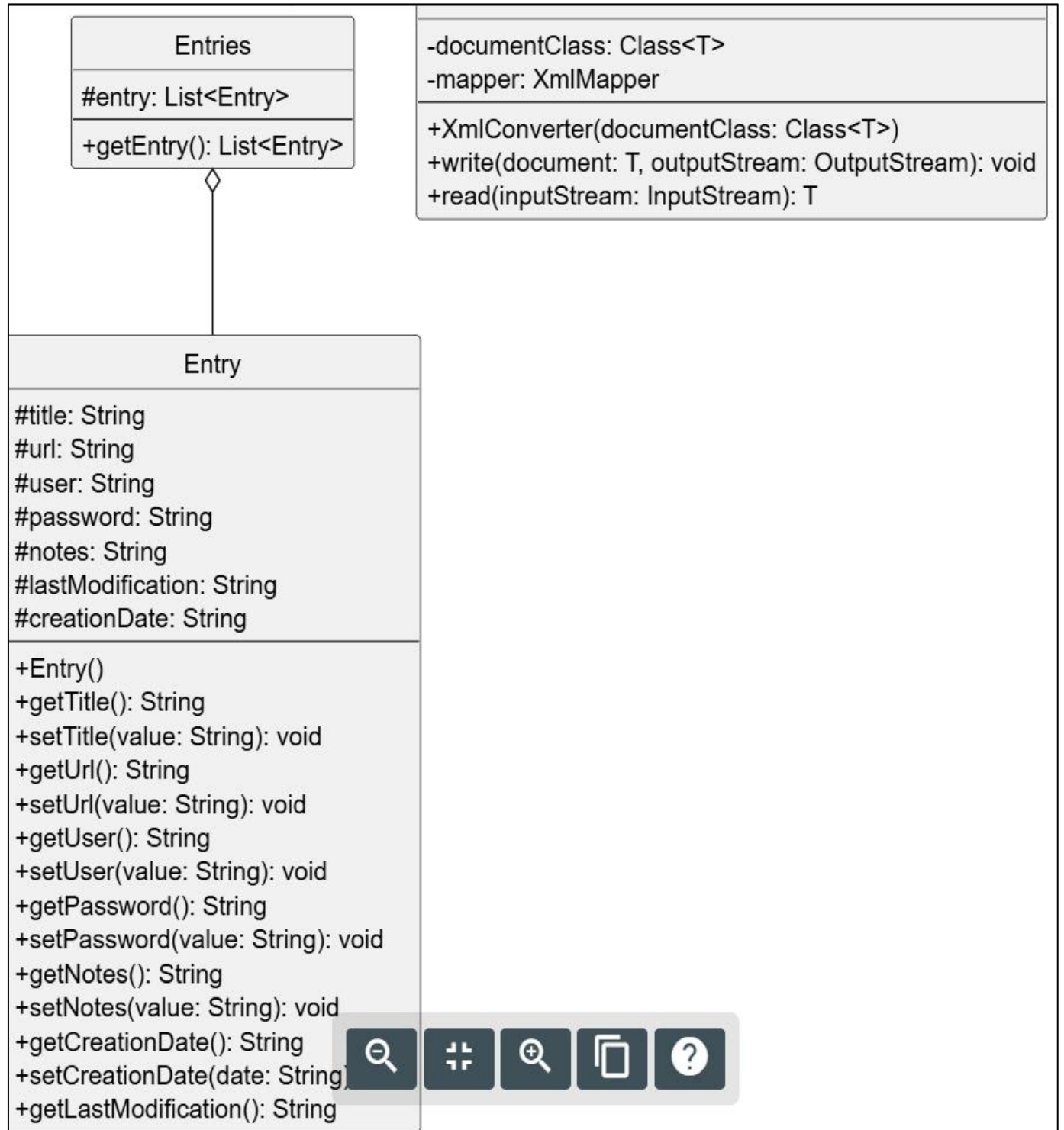
Semester: BSCYS-4B

Submitted To: Sir Zakaullah

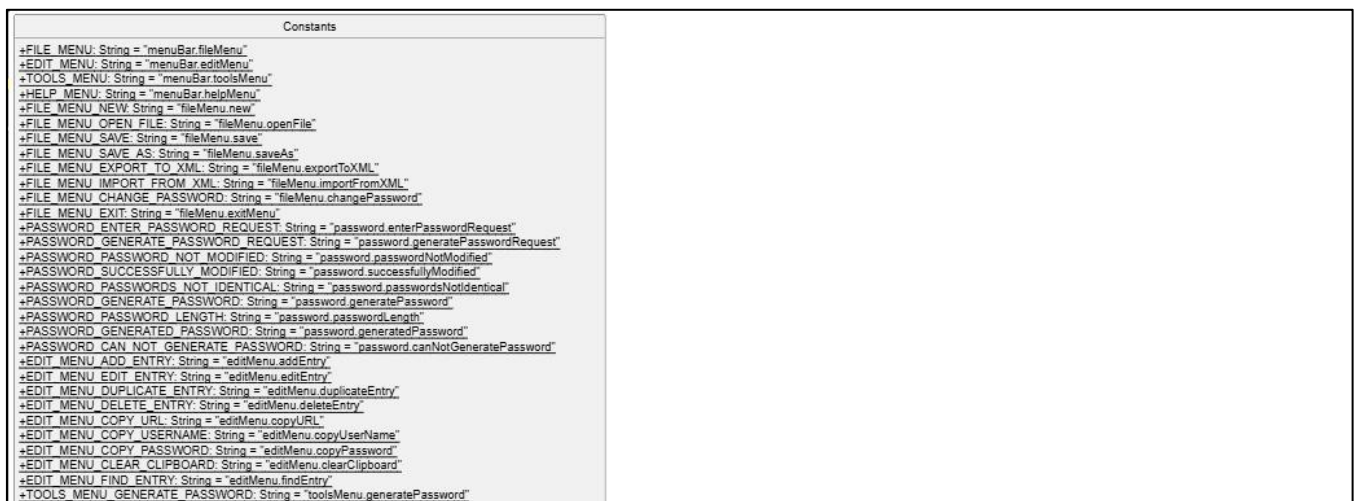
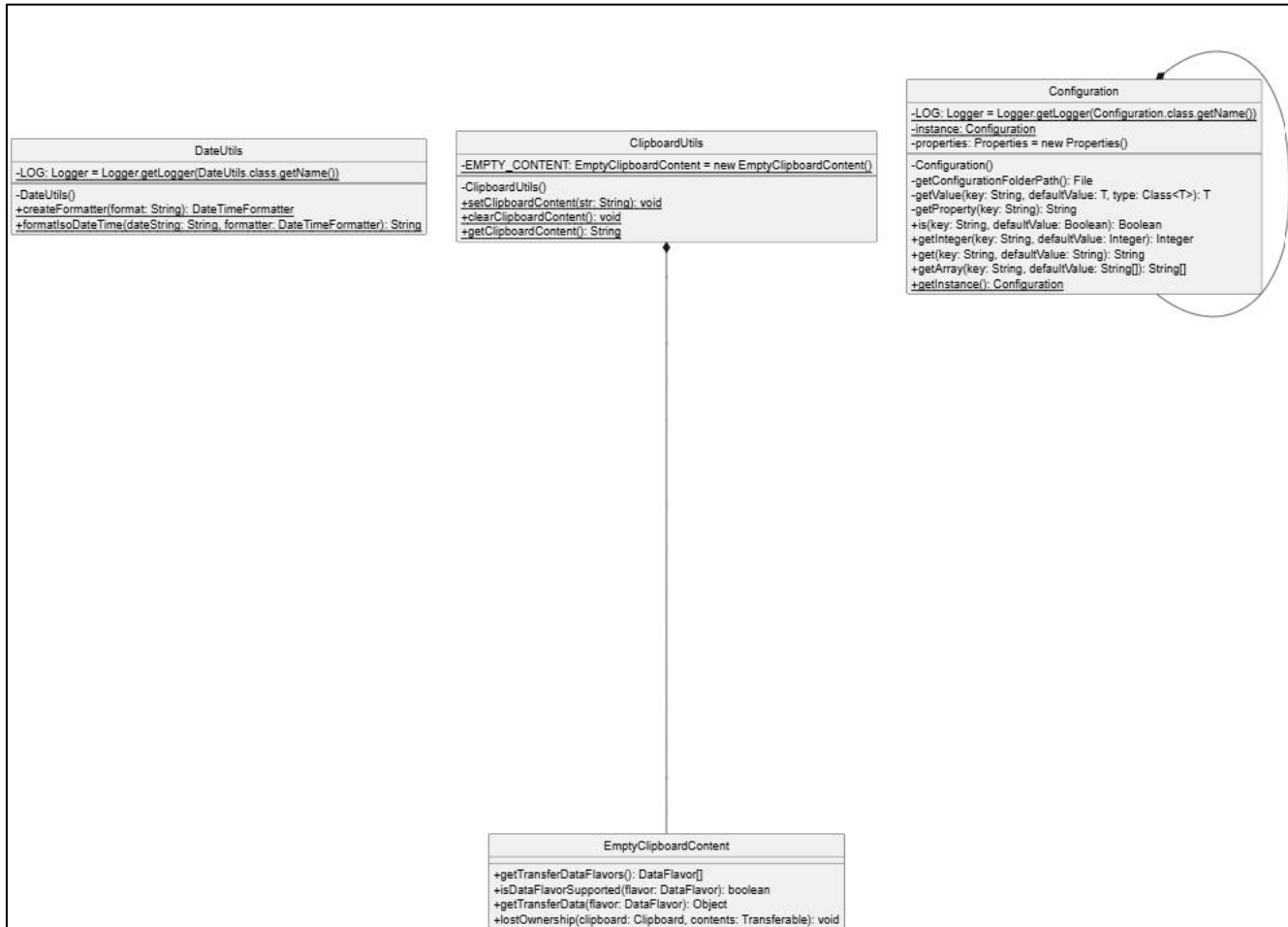
Date: March 12,2026

Class Diagrams:

- XML:



● UTIL:



```

+HELP_MENU_LICENSE: String = "helpMenu.license"
+HELP_MENU_ABOUT_JPASS: String = "helpMenu.aboutJpass"
+VIEW_WINDOW_TITLE: String = "viewWindow.title"
+VIEW_WINDOW_URL: String = "viewWindow.url"
+VIEW_WINDOW_USER: String = "viewWindow.user"
+VIEW_WINDOW_USER_NAME: String = "viewWindow.userName"
+VIEW_WINDOW_MODIFIED: String = "viewWindow.modified"
+VIEW_WINDOW_CREATED: String = "viewWindow.created"
+VIEW_WINDOW_PASSWORD: String = "viewWindow.password"
+VIEW_WINDOW_REPEAT: String = "viewWindow.repeat"
+VIEW_WINDOW_NOTES: String = "viewWindow.notes"
+VIEW_WINDOW_SETTINGS: String = "viewWindow.settings"
+VIEW_WINDOW_CUSTOM_SYMBOLS: String = "viewWindow.customSymbols"
+VIEW_WINDOW_UPPER_CASE_LETTERS: String = "viewWindow.upperCaseLetters"
+VIEW_WINDOW_LOWER_CASE_LETTERS: String = "viewWindow.lowerCaseLetters"
+VIEW_WINDOW_NUMBERS: String = "viewWindow.numbers"
+SHOW_MESSAGE_INFORMATION: String = "showMessage.information"
+SHOW_MESSAGE_CONFIRMATION: String = "showMessage.confirmation"
+SHOW_MESSAGE_WARNING: String = "showMessage.warning"
+SHOW_MESSAGE_ERROR: String = "showMessage.error"
+BUTTON_MESSAGE_YES: String = "buttonMessage.yes"
+BUTTON_MESSAGE_OK: String = "buttonMessage.ok"
+BUTTON_MESSAGE_ACCEPT: String = "buttonMessage.accept"
+BUTTON_MESSAGE_NO: String = "buttonMessage.no"
+BUTTON_MESSAGE_CANCEL: String = "buttonMessage.cancel"
+BUTTON_MESSAGE_CLOSE: String = "buttonMessage.close"
+ENTRY_DIALOG_SHOW_ENTRY: String = "entryDialog.showEntry"
+ENTRY_DIALOG_GENERATE_ENTRY: String = "entryDialog.generateEntry"
+ENTRY_DIALOG_COPY_ENTRY: String = "entryDialog.copyEntry"
+ENTRY_DIALOG_FILL_TITLE_FIELD: String = "entryDialog.fillTitleField"
+ENTRY_DIALOG_TITLE_ALREADY_EXISTS: String = "entryDialog.titleAlreadyExists"
+ENTRY_DIALOG_PLEASE_SELECT_ENTRY: String = "entryDialog.pleaseSelectEntry"
+ENTRY_DIALOG_WANT_DELETE_ENTRY: String = "entryDialog.wantDeleteEntry"
+ENTRY_DIALOG_DUPLICATE_ENTRY: String = "entryDialog.duplicateEntry"
+ENTRY_DIALOG_EDIT_ENTRY: String = "entryDialog.editEntry"
+ENTRY_DIALOG_ADD_NEW_ENTRY: String = "entryDialog.addNewEntry"
+BOTTOM_MENU_ENTRIES_COUNT: String = "bottomMenu.entriesCount"
+BOTTOM_MENU_ENTRIES_FOUND: String = "bottomMenu.entriesFound"
+PANEL_PROCESSING: String = "panel.processing"
+PANEL_FIND: String = "panel.find"
+PANEL_EXPORT: String = "panel.export"
+PANEL_IMPORT: String = "panel.import"
+PANEL_SAVE: String = "panel.save"
+PANEL_OPEN: String = "panel.open"
+PANEL_SAVE_MODIFIED_QUESTION_MESSAGE: String = "panel.saveModifiedQuestionMessage"
+PANEL_UNENCRYPTED_DATA_WARNING_MESSAGE: String = "panel.unencryptedDataWarningMessage"

-Constants()

```

CryptUtils

```

-CryptUtils()
+getPBKDF2KeyWithDefaultIterations(text: char[], salt: byte[]): byte[]
+getPBKDF2Key(text: char[], salt: byte[], iteration: int): byte[]
+getSha256HashWithDefaultIterations(text: char[]): byte[]
+getSha256Hash(text: char[]): byte[]
+getSha256Hash(text: char[], iteration: int): byte[]
+generateRandomSalt(saltLength: int): byte[]
+newRandomNumberGenerator(): Random

```

SpringUtilities

```

-SpringUtilities()
+makeGrid(parent: Container, rows: int, cols: int, initialX: int, initialY: int, xPad: int, yPad: int): void
+getConstraintsForCell(row: int, col: int, parent: Container, cols: int): SpringLayout.Constraints
+makeCompactGrid(parent: Container, rows: int, cols: int, initialX: int, initialY: int, xPad: int, yPad: int): void

```

StringUtils

```

-StringUtils()
+stripNonValidXMLCharacters(in: String): String
+stripString(text: String): String
+stripString(text: String, length: int): String

```

● Action:

AbstractMenuAction +AbstractMenuAction(text: String, icon: Icon, accelerator: KeyStroke)	CloseListener +windowClosing(event: WindowEvent): void	TableListener +mouseClicked(evt: MouseEvent): void +mousePressed(evt: MouseEvent): void +mouseReleased(evt: MouseEvent): void -checkPopup(evt: MouseEvent): void
TextComponentAction +TextComponentAction(text: String, accelerator: KeyStroke, mnemonic: int) +isEnabled(component: JTextComponent): boolean	TextComponentPopupListener -popup: JPopupMenu -items: Map<TextComponentActionType, JMenuItem> +TextComponentPopupListener() -showPopupMenu(e: MouseEvent): void +mousePressed(e: MouseEvent): void +mouseReleased(e: MouseEvent): void	Worker -parent: JPassFrame +Worker(parent: JPassFrame) #done(): void #showErrorMessage(e: Exception): void #stopProcessing(): void

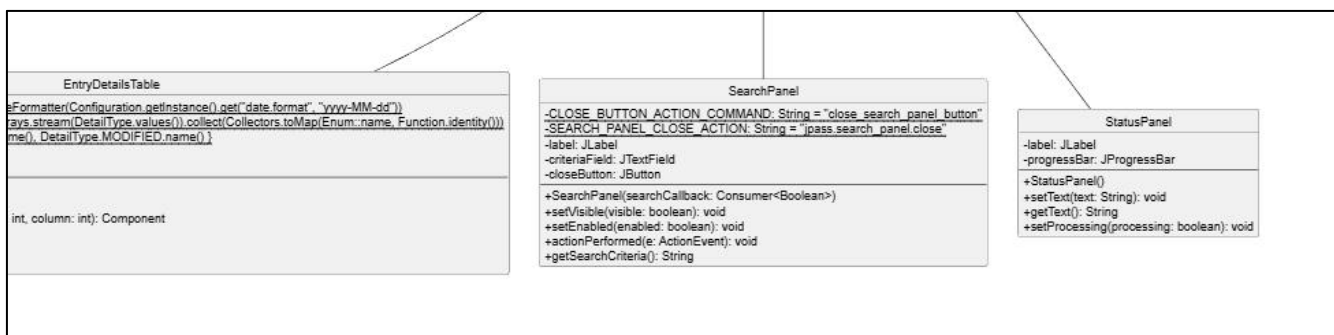
```

classDiagram
    class Worker {
        -parent: JPassFrame
        +Worker(parent: JPassFrame)
        #done(): void
        #showErrorMessage(e: Exception): void
        #stopProcessing(): void
    }
    class EntryDialog {
        -NULL_ECHO: char = '\0'
        -fieldPanel: JPanel
        -notesPanel: JPanel
        -buttonPanel: JPanel
        -passwordButtonPanel: JPanel
        -titleLabel: JTextField
        -textField: JTextField
        -passwordField: JPasswordField
        -repeatField: JPasswordField
        -urlField: JTextField
        -notesField: JTextArea
        -okButton: JButton
        -cancelButton: JButton
        -showButton: JToggleButton
        -generateButton: JButton
        -copyButton: JButton
        -originalEcho: char
        -modifiedEntry: Entry
        -newEntry: boolean
        -originalTitle: String
        +EntryDialog(parent: JPassFrame, title: String, entry: Entry, newEntry: boolean)
        +actionPerformed(e: ActionEvent): void
        +fillDialogFromEntry(entry: Entry): void
        +getEntryFromDialog(): Entry
        +getModifiedEntry(): Optional<Entry>
        +checkEntryTitle(): boolean
    }
    class JPassFrame {
        -localizedMessages: ResourceBundle
        -LOG: Logger = Logger.getLogger(JPassFrame.class.getName())
        -instance: JPassFrame
        +PROGRAM NAME: String = "JPass Password Manager"
        +PROGRAM VERSION: String = "1.0.7-SNAPSHOT"
        -popup: JPopupMenu
        -topContainerPanel: JPanel
        -jpassMenuBar: JMenuBar
        -searchPanel: SearchPanel
        -fileMenu: JMenu
        -editMenu: JMenu
        -toolsMenu: JMenu
        -helpMenu: JMenu
        -toolBar: JToolBar
        -scrollPane: JScrollPane
        -entryDetailsTable: EntryDetailsTable
        -model: DataModel = DataModel.getInstance()
        -statusPanel: StatusPanel
        -processing: boolean = false
        -JPassFrame(fileName: String, locale: Locale)
        +getInstance(): JPassFrame
        +getInstance(fileName: String): JPassFrame
        +getEntryTitleTable(): JTable
        +getModel(): DataModel
        +clearModel(): void
        +refreshFrameTitle(): void
        +refreshEntryTitleList(selectTitle: String): void
        +refreshAll(): void
        +exitFrame(): void
        +getPopup(): JPopupMenu
        +setProcessing(processing: boolean): void
        +isProcessing(): boolean
        +getSearchPanel(): SearchPanel
        +getLocalizedMessages(): ResourceBundle
    }
    class MessageDialog {
        +getMessageDialog(MessageDialog class getName()): MessageDialog
        N: int = -1
        I: int = 0
        L_OPTION: int = 1
        TION: int = 2
        O: int = 0
        I: int = 1
        S: int = 2
        U: int = -1
        +MessageDialog(message: Object, title: String, icon: ImageIcon, optionType: int)
        +MessageDialog(message: Object, title: String, icon: ImageIcon, optionType: int)
        +MessageDialog(component: Component, message: Object, title: String, icon: ImageIcon, optionType: int): void
        +MessageDialog(title: String, icon: ImageIcon): void
        +MessageDialog(actionEvent: ActionEvent): void
        +MessageDialog(parentComponent: Component, message: Object, title: String, icon: ImageIcon): void
        +MessageDialog(parentComponent: Component, message: Object, title: String, icon: ImageIcon, optionType: int): void
        +MessageDialog(parentComponent: Component, message: String): void
        +MessageDialog(parentComponent: Component, message: String): void
        +MessageDialog(parentComponent: Component, message: String): void
        +MessageDialog(parentComponent: Component, message: String, optionType: int): void
        +MessageDialog(parentComponent: Component, confirm: boolean): void
        +MessageDialog(icon: ImageIcon)
        +MessageDialog(x: int, width: int, height: int): ImageIcon
        +MessageDialog(fileName: String): String
        +MessageDialog(title: String, textField: String): void
    }
    class EntryHelper {
        -EntryHelper()
        +deleteEntry(parent: JPassFrame): void
        +duplicateEntry(parent: JPassFrame): void
        +editEntry(parent: JPassFrame): void
        +addEntry(parent: JPassFrame): void
        +getSelectedEntry(parent: JPassFrame): Entry
        +copyEntryField(parent: JPassFrame, content: String): void
    }
    class FileHandler {
        -OPEN_ERROR_CHECK_PASSWORD_ERROR_MESSAGE: String
        The password might be incorrect\n(Error: %s)\n-CREATE FILE QUESTION MESSAGE: String = "File %s\nDo you want to create the file?"\n-OPERATION ERROR MESSAGE: String = "An error occurred while creating the file.\n%s"\n-FILE_OVERWRITE_QUESTION_MESSAGE: String = "The file %s already exists. Do you want to overwrite?"\n-JPASS_DATA_FILES: String = "JPASS Data Files (*.jpas*)\n-XMl FILES: String = "XMl Files (*.xml)"\n-FileHelper()\n+createNew(parent: JPassFrame): void\n+exportFile(parent: JPassFrame): void\n+importFile(parent: JPassFrame): void\n+importFileInBackground(fileName: String, parent: JPassFrame): void\n+saveFile(parent: JPassFrame, saveAs: boolean): void\n+saveFileInBackground(fileName: String, parent: JPassFrame): void\n+openFile(parent: JPassFrame): void\n+openFileInBackground(fileName: String, parent: JPassFrame): void\n+handleFileNotFound(parent: JPassFrame, fileName: String): void\n+showFileChooser(parent: JPassFrame, taskName: String): void\n+checkFileOverwrite(fileName: String, parent: JPassFrame): void\n+checkExtension(fileName: String, extension: String): boolean
    }
    Worker --> JPassFrame
    EntryDialog --> JPassFrame
    EntryHelper ..> JPassFrame
    FileHandler ..> JPassFrame

```

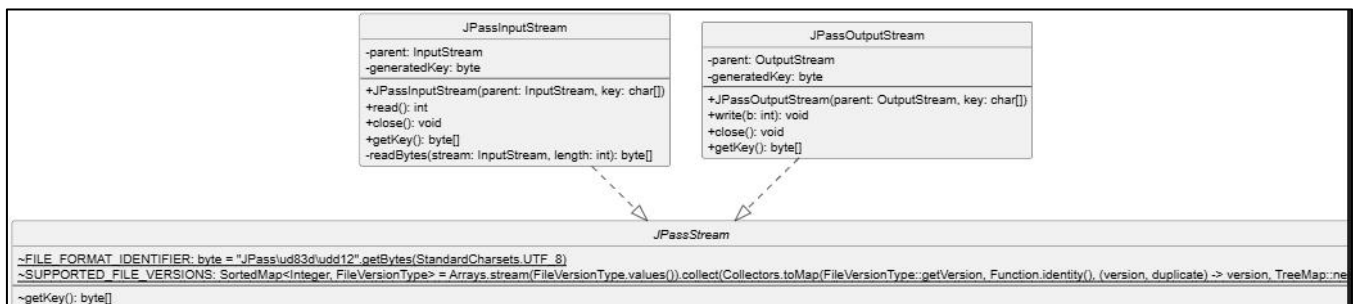
The diagram illustrates the architecture of the JPass application. It features several key components:

- JPassFrame**: The main window containing localized messages, logging, program information, UI elements like menus and toolbars, a search panel, a scroll pane for entry details, a status panel, and processing logic.
- Worker**: A helper class responsible for handling exceptions and stopping processing.
- EntryDialog**: A dialog box for managing entries, including fields for name, password, URL, notes, and buttons for adding, editing, deleting, and generating passwords.
- MessageDialog**: A utility class for displaying various types of messages (info, warning, error) with icons and options.
- EntryHelper**: Provides methods for performing actions on entries such as deletion, duplication, editing, addition, selection, and copying.
- FileHandler**: Manages file operations, including saving, loading, overwriting, and checking file extensions.

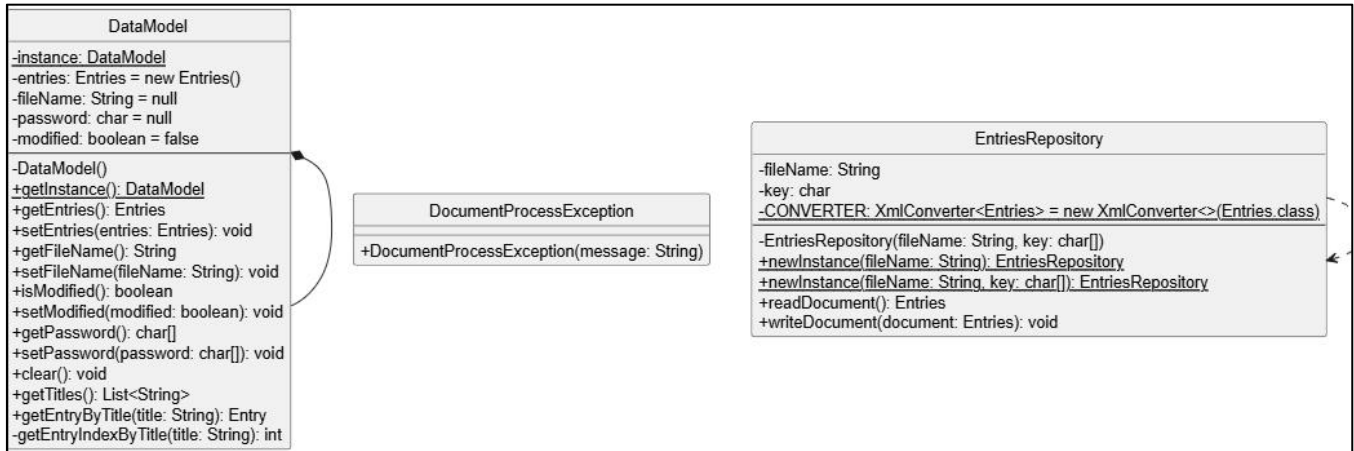




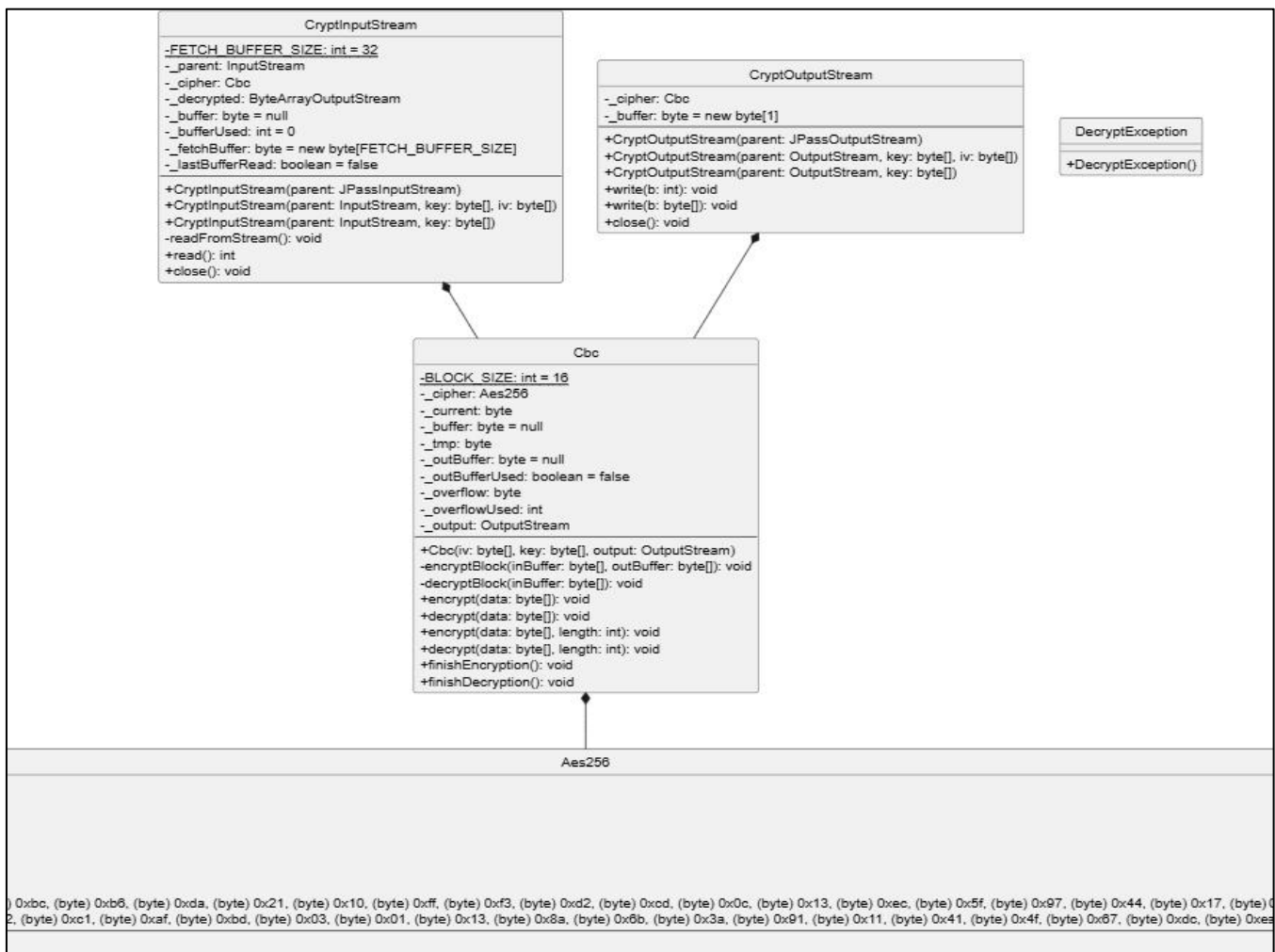
● 10:



● Data:



● Crypt:



● Relationship:

1. Cbc → Aes256

Composition Cbc owns Aes256. Aes256 cannot exist without Cbc. Cbc uses it internally for encryption and decryption.

2. CryptInputStream → Cbc

Composition CryptInputStream owns a Cbc object. It uses it to decrypt incoming data streams.

3. CryptInputStream → JPassInputStream

Dependency CryptInputStream depends on JPassInputStream to read raw encrypted bytes from file.

4. CryptOutputStream → Cbc

Composition CryptOutputStream owns a Cbc object. It uses it to encrypt outgoing data streams.

5. CryptOutputStream → JPassOutputStream

Dependency CryptOutputStream depends on JPassOutputStream to write encrypted bytes to file.

6. JPassInputStream → JPassStream

Realization JPassInputStream implements the JPassStream interface, providing concrete reading functionality.

7. JPassOutputStream → JPassStream

Realization JPassOutputStream implements the JPassStream interface, providing concrete writing functionality.

8. DataModel → Entries

Composition DataModel owns the Entries object. Entries cannot exist independently outside DataModel.

9. DataModel → Entry

Dependency DataModel uses Entry objects to search, fetch, and manage individual password records.

10. Entries → Entry

Aggregation Entries holds a list of Entry objects. Entry can exist independently of Entries.

11. EntriesRepository → XmlConverter

Composition EntriesRepository owns XmlConverter. It uses it to read and write XML data.

12. EntriesRepository → Entries

Aggregation EntriesRepository reads and writes Entries objects. Entries exists independently of the repository.

13. JPassFrame → DataModel

Composition JPassFrame owns the DataModel. It is the core data source of the main application window.

14. JPassFrame → SearchPanel

Composition JPassFrame owns SearchPanel. SearchPanel is a UI component built inside the main window.

15. JPassFrame → EntryDetailsTable

Composition JPassFrame owns EntryDetailsTable. It is part of the main window UI to display entries.

16. JPassFrame → StatusPanel

Composition JPassFrame owns StatusPanel. It shows processing status at the bottom of the main window.

17. Worker → JPassFrame

Dependency Worker depends on JPassFrame as its parent. It performs background tasks and reports back to the frame.

18. EntryDialog → JPassFrame

Dependency EntryDialog depends on JPassFrame as its parent window to open and display correctly.

19. EntryDialog → Entry

Composition EntryDialog owns an Entry object. It holds the entry being created or edited by the user.

20. EntryDetailsTable → Entry

Dependency EntryDetailsTable depends on Entry to display its data in rows inside the table.

21. EntryHelper → JPassFrame

Dependency EntryHelper depends on JPassFrame to perform add, edit, delete, and duplicate entry operations.

22. EntryHelper → Entry

Dependency EntryHelper depends on Entry to manage individual password records.

23. FileHelper → JPassFrame

Dependency FileHelper depends on JPassFrame to perform file operations like open, save, import, and export.

24. FileHelper → EntriesRepository

Dependency FileHelper depends on EntriesRepository to read and write the password data file.

25. JPass → JPassFrame

Dependency JPass is the main entry point of the application. It depends on JPassFrame to launch the UI.

● Analysis For Weaknesses:

1. Password Stays in Memory Too Long

Where: DataModel class

Master password is stored as char[] and never cleared after use. A memory dump can expose it easily.

2. Anyone Can Access All Data

Where: DataModel class

Singleton pattern means any class can call getInstance() and access all passwords without any permission check.

3. Individual Passwords Stored as Plain Text

Where: Entry class

Every saved password is a plain String in memory. Java Strings cannot be manually cleared, so they stay readable until garbage collection.

4. Export File May Be Unencrypted

Where: XmlConverter class

XmlConverter writes directly to output stream with no visible encryption. The exported XML file can be plain text on disk, readable by anyone.

5. Key Stored in the Same File as Data

Where: JPassInputStream class

getKey() reads the encryption key from the same file as the encrypted data. If both are in one file, encryption provides almost no protection.

6. No In-Memory Encryption

Where: DataModel class

All entries are stored as plain Entries objects in RAM with no in-memory encryption. Anyone with RAM access can read all passwords directly.

7. Clipboard Never Auto Cleared

Where: ClipboardUtils class

After copying a password, there is no timer to clear the clipboard. It stays there indefinitely and any other application on the system can silently read it.

8. No Auto Lock When App is Left Open

Where: JPassFrame class

No session timeout or auto-lock feature exists. If a user walks away with the app open, anyone can sit down and access everything freely.

9. Weak Password Generator

Where: GeneratePasswordDialog class

Basic Java Random is used which is not cryptographically secure. Generated passwords can be predicted. CryptUtils already has a secure random method available but it is not being used here.

10. No File Integrity Check

Where: FileHelper class

Before loading a file, there is no verification that it has been tampered with. A modified or corrupted file loads silently without any warning.

11. No Re-Authentication Before Editing Entries

Where: EntryHelper and EntryDialog classes

Anyone can edit, delete or duplicate entries without re-entering the master password. Once the app is open, everything is freely accessible.

12. XML Import Without Validation

Where: FileHelper class

The importFile() method imports XML files without validating their content. A malicious XML file with harmful or fake data can be imported without any check.

13. No Security Layer Between UI and Data

Where: JPassFrame and DataModel classes

The UI layer directly accesses DataModel with no middle security layer. Any UI component can reach sensitive data without restriction.

14. IV Reuse Possible in Encryption

Where: CryptInputStream and CryptOutputStream classes

No Initialization Vector validation is visible in the diagram. If the same IV is reused across encryption sessions, the encryption becomes significantly weaker.

15. No Audit Trail or Logging

Where: Entire project

No class shows any audit logging. Failed login attempts, file access events, and password changes are not recorded anywhere. Suspicious activity cannot be detected.

16. Exception Messages Can Leak Internal Information

Where: DocumentProcessException class

Error messages may contain internal file structure details, encryption method names, or system paths that are useful for an attacker.

17. Search Criteria Not Protected

Where: SearchPanel class

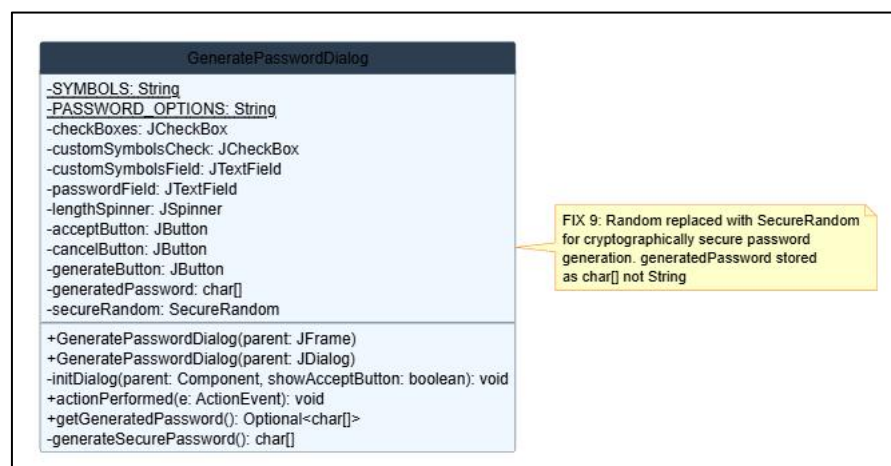
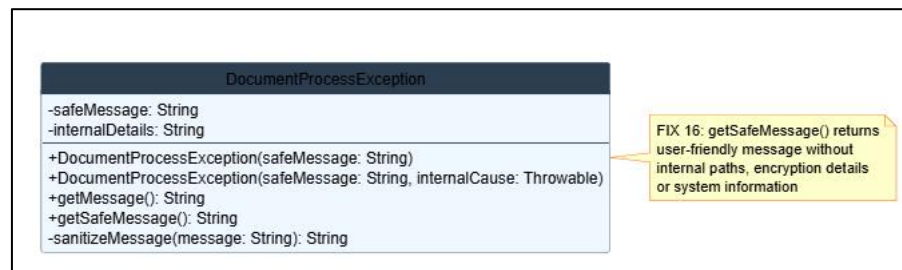
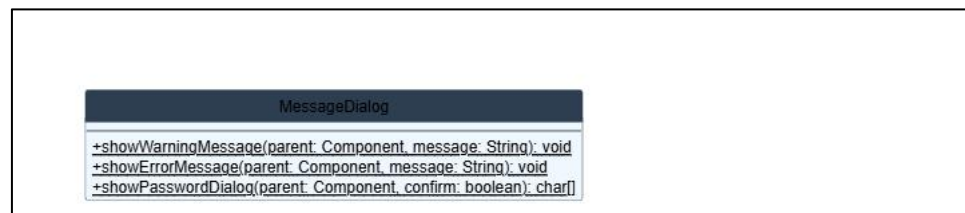
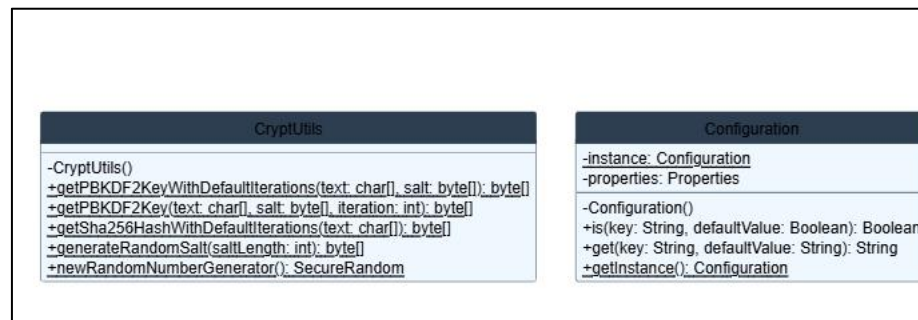
Search input is processed as plain text. If search history is stored anywhere, it could partially expose sensitive entry titles or usernames.

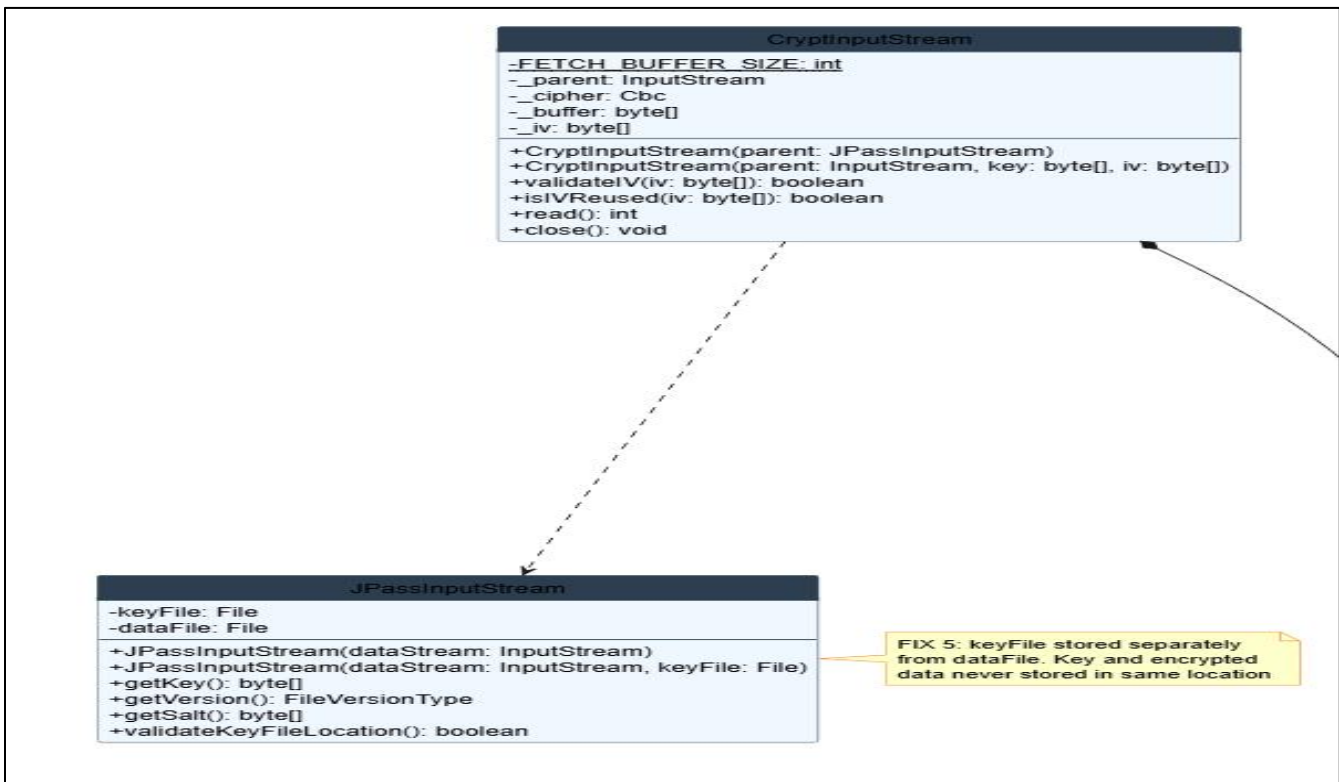
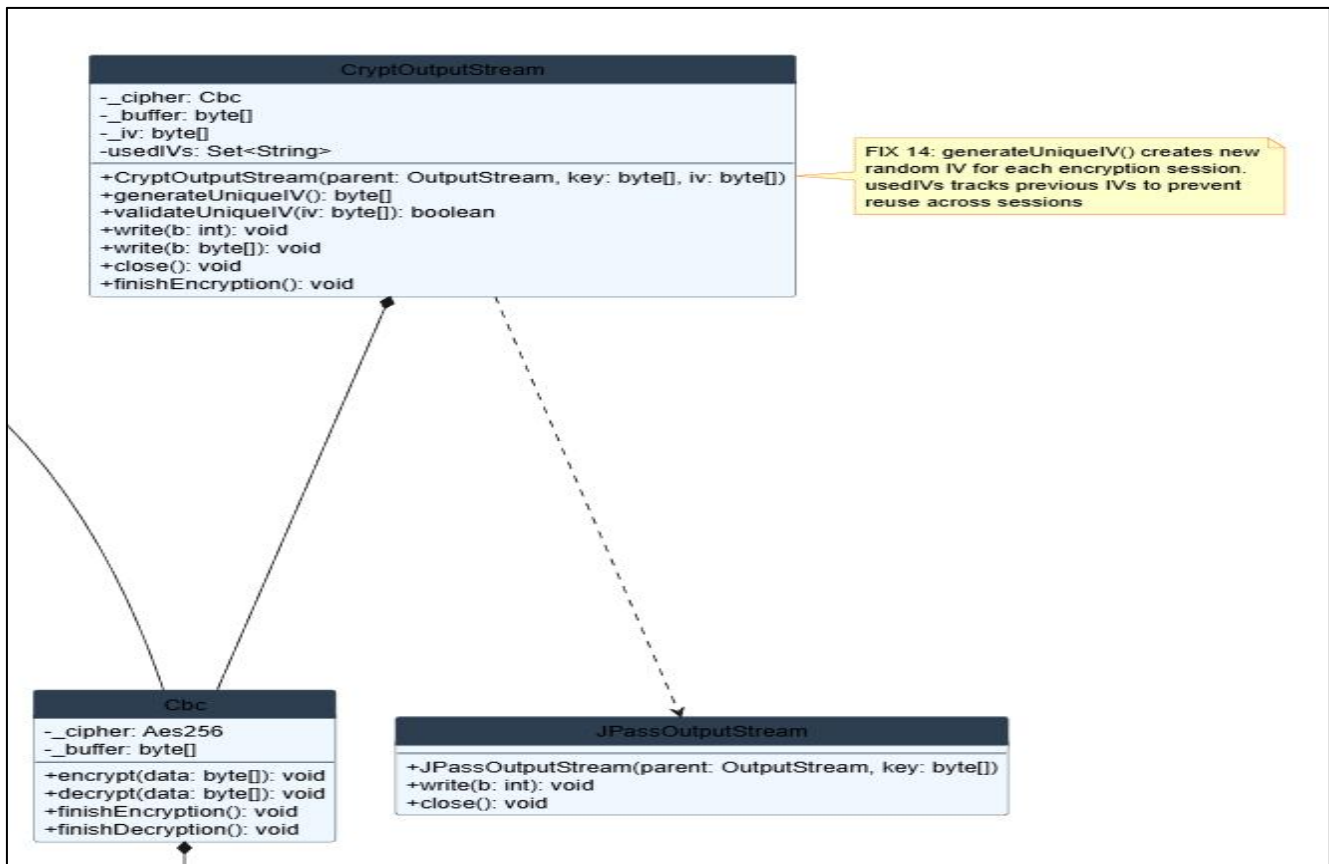
18. Worker Class Has No Authentication Check

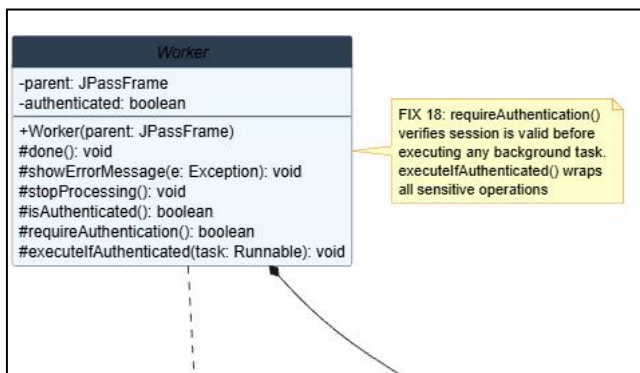
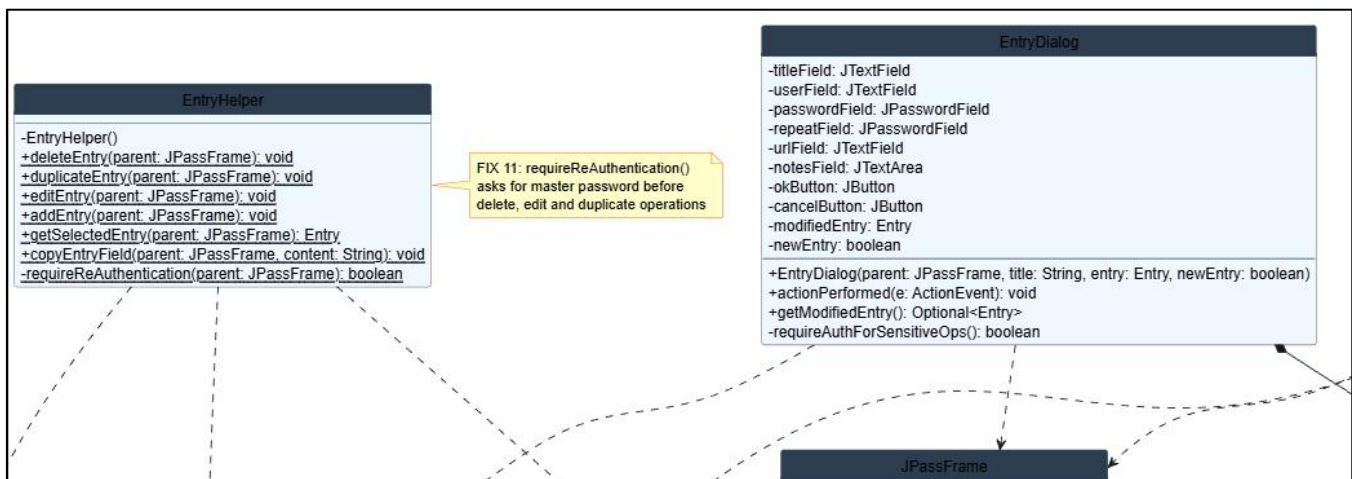
Where: Worker class

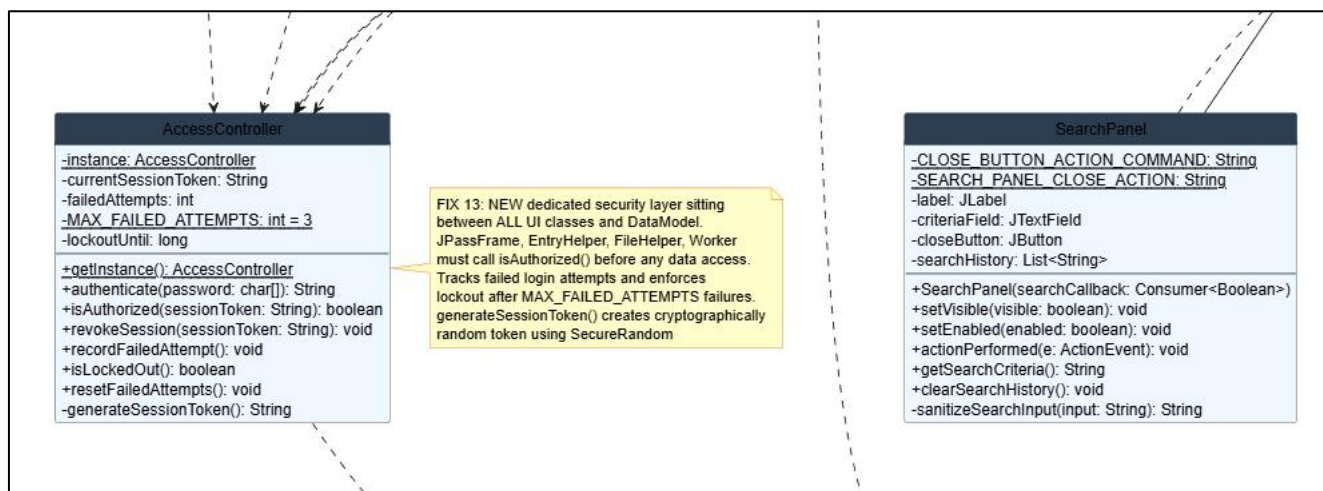
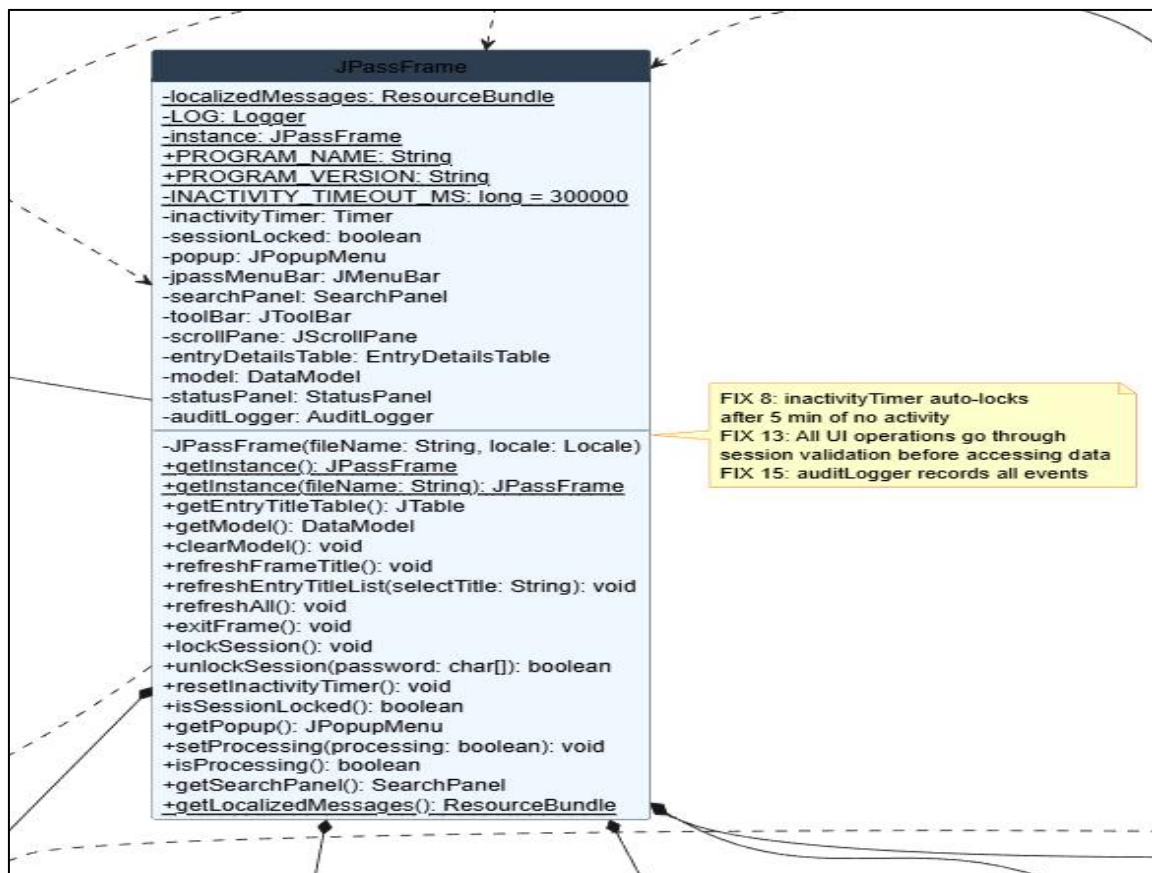
Worker performs background operations using a reference to JPassFrame but has no authentication check before executing sensitive tasks.

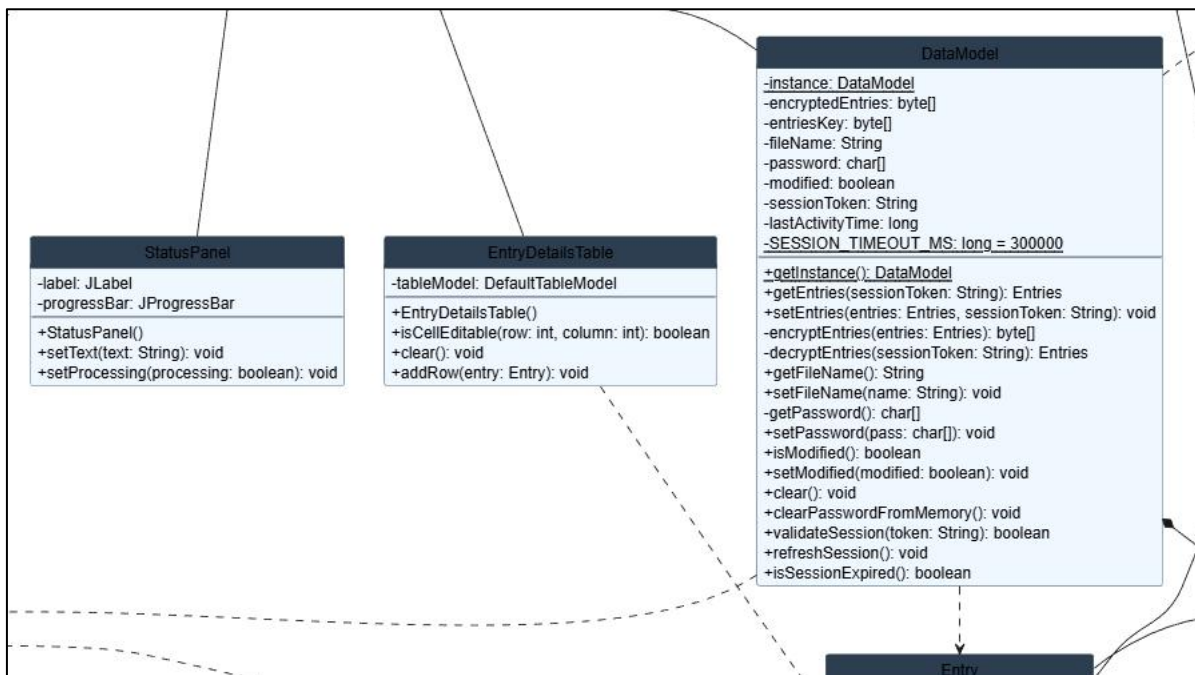
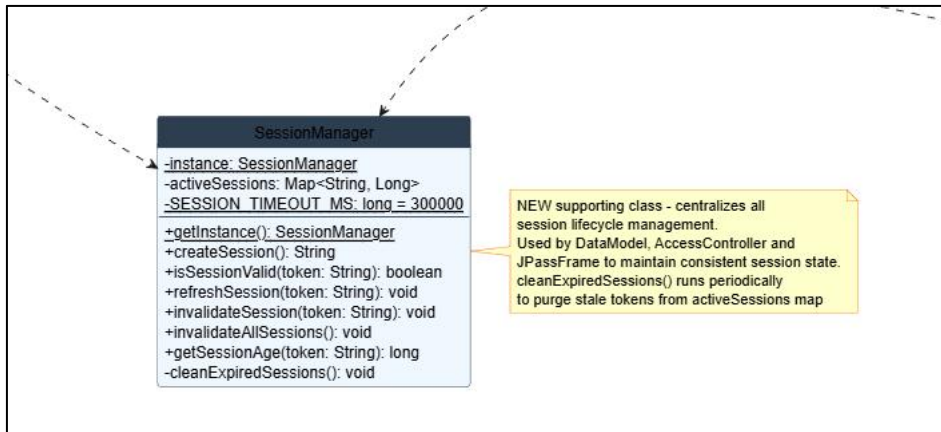
Secure Class Diagram

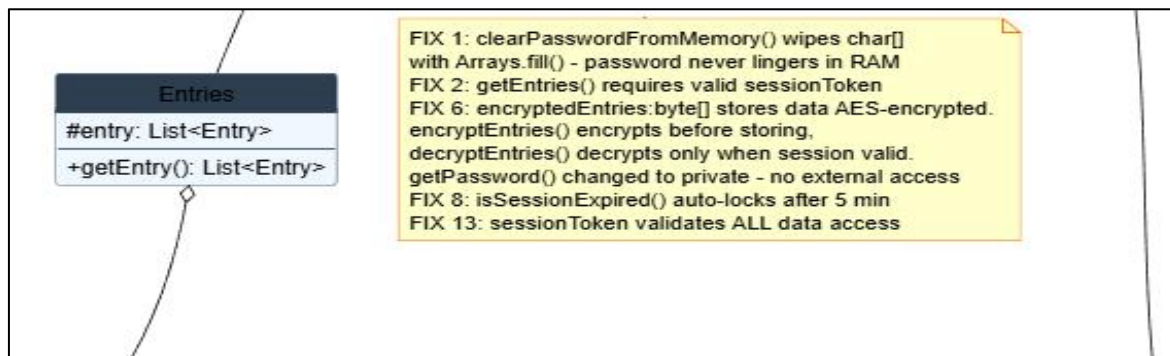
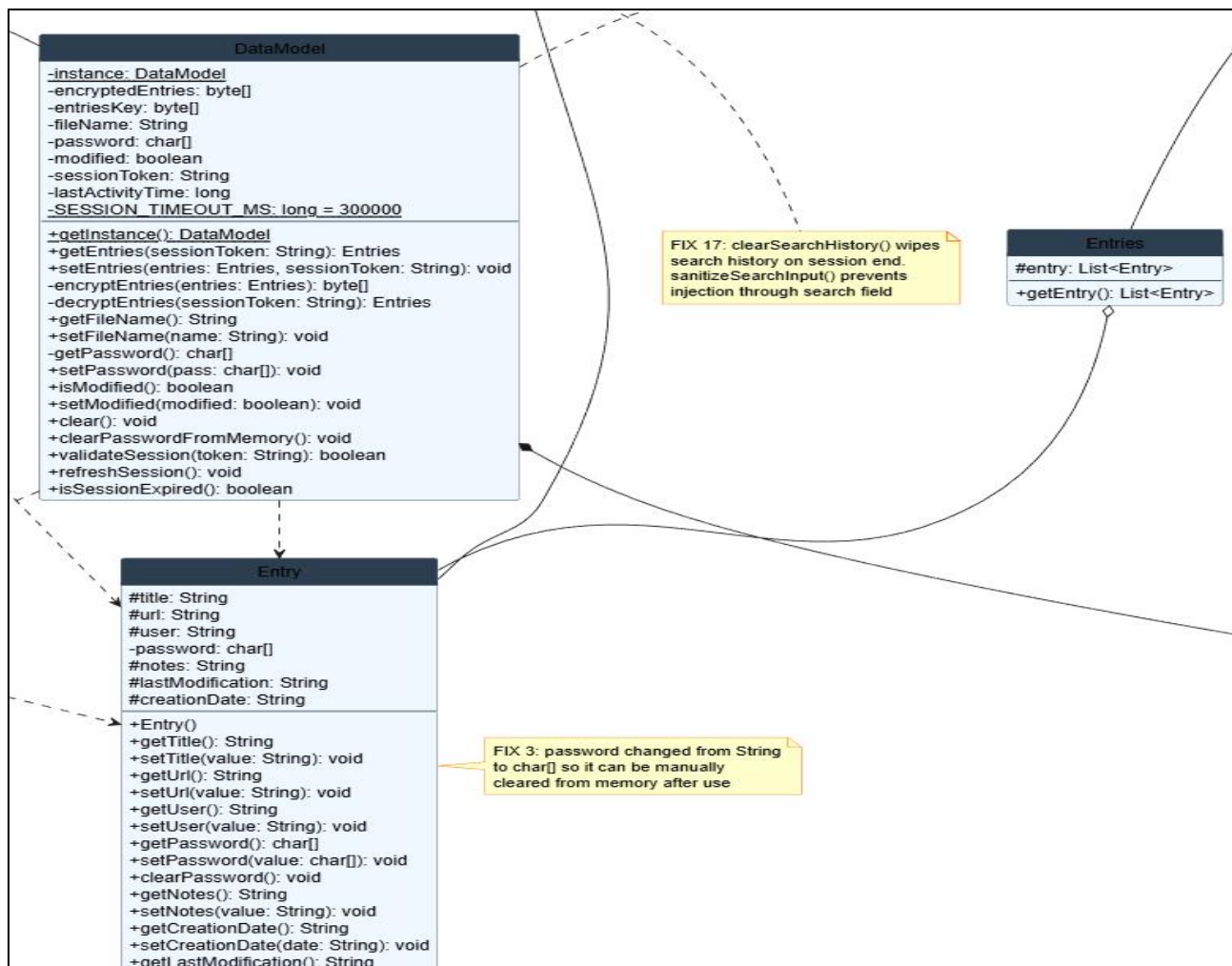


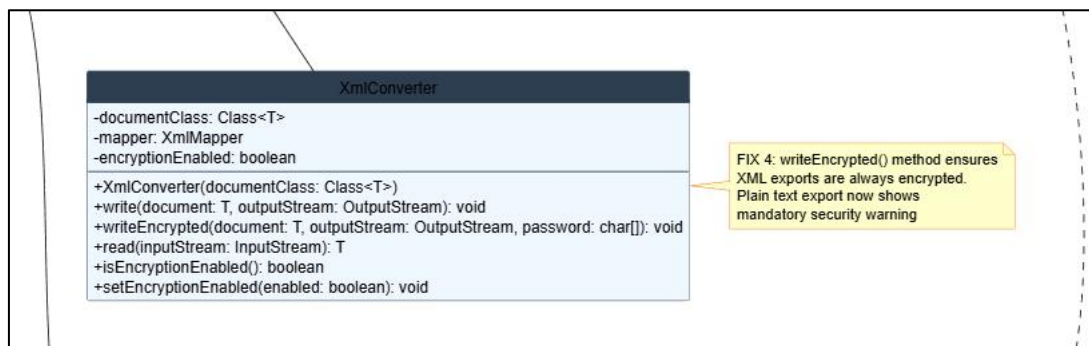
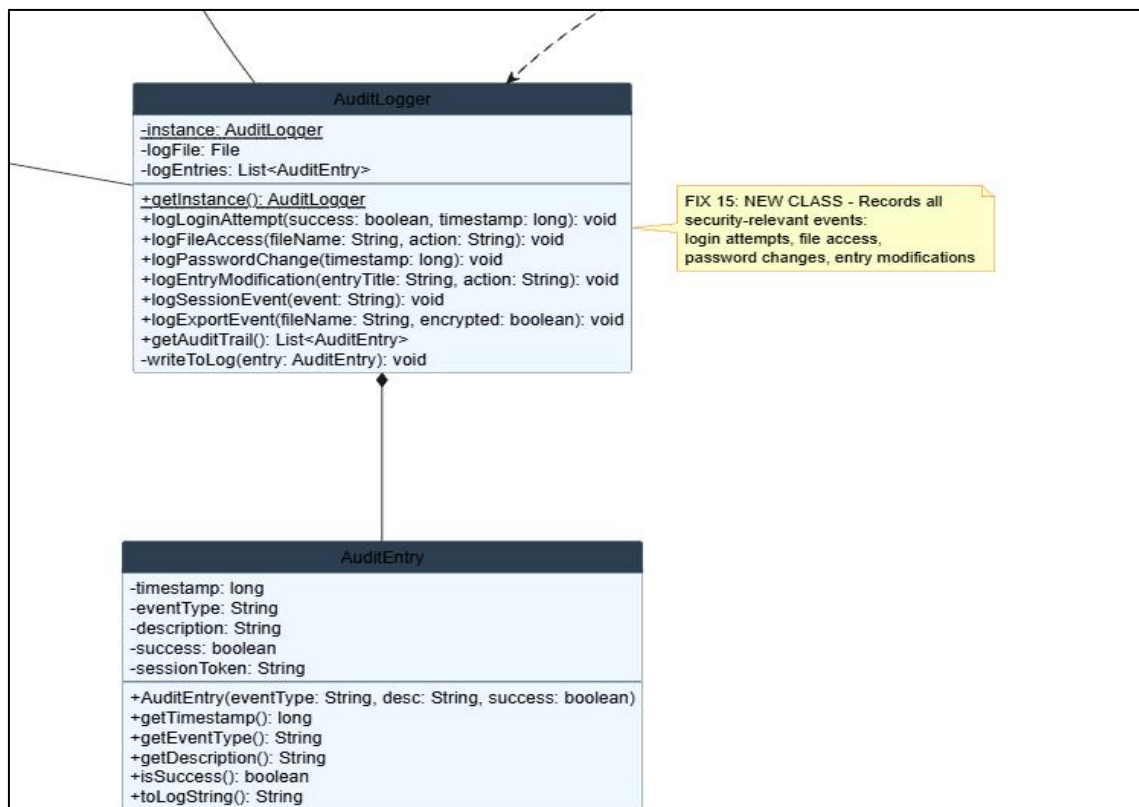


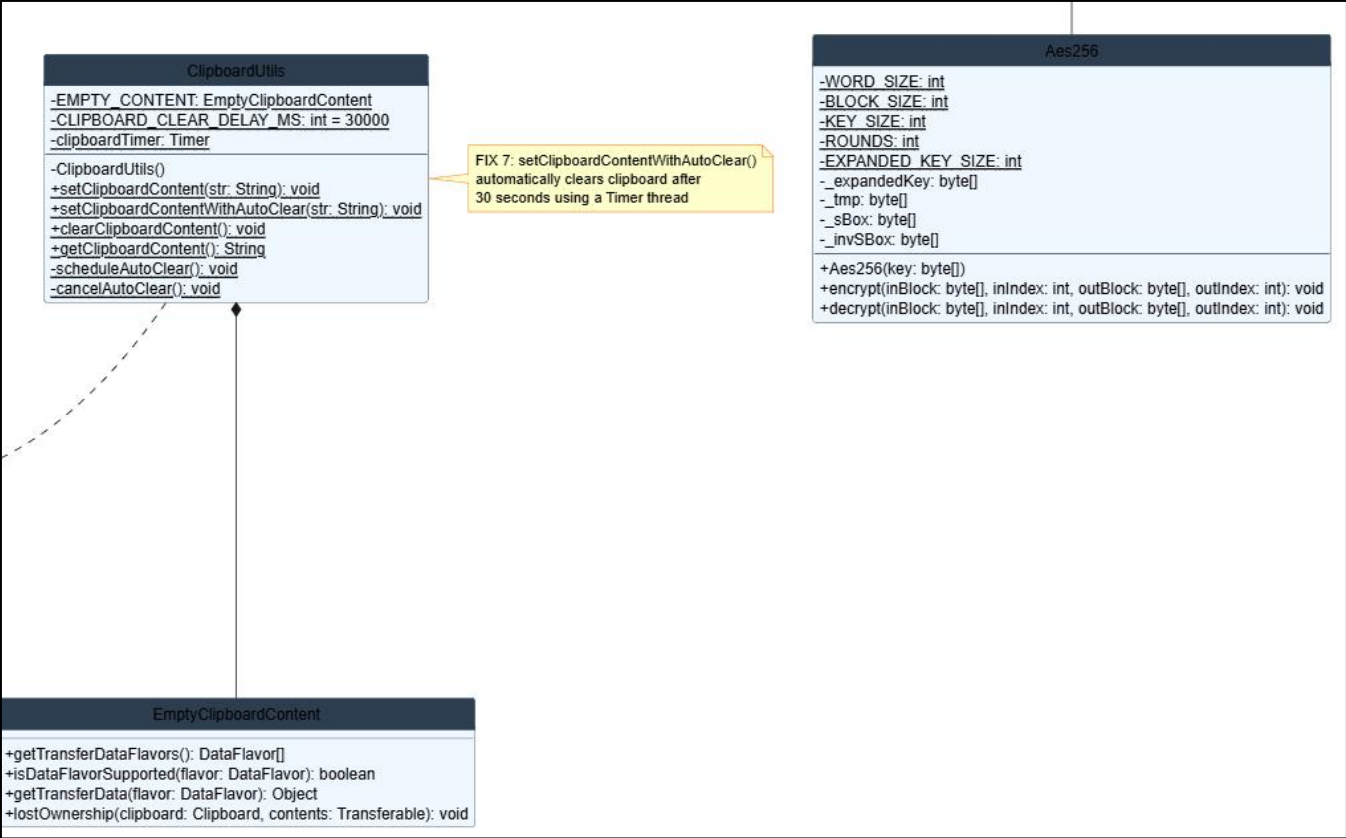












Following changes are made in code:

- **DataModel.java — Password Memory Erasure**

Old Code: *this.password = null* inside **clear()** and **setPassword()**

New Code: *Arrays.fill(this.password, '\0')* then **null**

Why: Setting a Java reference to null only removes the pointer. The actual **char[]** bytes remain alive in heap memory until the Garbage Collector (GC) runs, which could be minutes later in a long-running application. A heap dump (*jmap -dump*) or operating system crash dump taken during that window exposes the master password in plaintext. *Arrays.fill()* immediately overwrites every byte of the **char[]** in-place, so the bytes are gone before any heap snapshot can capture them.

- **Entry.java — Password Access Layer**

Old Code: *String getPassword()*-String is immutable and can never be zeroed

New Code: *getPasswordAsChars()* returns **char[]** (caller zeros it after use) + *clearPassword()* nulls the reference.

Why: Java **String** bytes cannot be zeroed because Strings are immutable and interned by the JVM. Once created, their backing **char[]** is frozen and inaccessible to application code. A **char[]** is mutable and can be completely overwritten with *Arrays.fill()* immediately after use, closing the window during which the plaintext password bytes exist at a known heap address.

- **ClipboardUtils.java — Auto-Clear Clipboard**

Old Code: *setClipboardContent()*-left the password on the system clipboard indefinitely

New Code: *setClipboardContentWithAutoClear()* schedules a **Timer** to call *clearClipboardContent()* after 30 seconds.

Why: Any application running on the same desktop session (browser extensions, screenshot tools, cloud clipboard synchronization services, or malware) can read the system clipboard at any time without user interaction. Leaving a password on the clipboard for hours or days greatly expands the attack window. The 30-second auto-clear timeout matches the defaults used by 1Password and Bitwarden, which are industry-standard password managers.

- **DataModel.java + JPassFrame.java — Inactivity Session Lock**

Old Code: Application stayed open forever after login, no idle timeout

New Code: *DataModel* tracks *lastActivityTime*, *isSessionExpired()* returns true after 5 minutes.

JPassFrame uses a **javax.swing.Timer** (fires every 30s) and an *AWTEventListener* that resets the clock on every mouse/key event. *lockSession()* clears memory and prompts for password re-entry.

Why: An unattended machine with JPass open gives any physical attacker unlimited time to read every stored credential without needing the master password. The 5-minute inactivity timeout matches KeyPass defaults and most enterprise security policies for credential management tools. The implementation uses a Swing Timer (not `java.util.Timer`) so all callbacks fire on the Event Dispatch Thread, making it safe to update the UI directly.

- **CryptUtils.java + GeneratePasswordDialog.java — SecureRandom**

Old Code: Return type was *Random* with silent fallback to *new Random()* if *SecureRandom* threw.

Generated password stored as String. `Optional<String>` returned.

New Code: Return type changed to *SecureRandom*; fallback removed (fails loudly).

GeneratePasswordDialog uses *private final SecureRandom secureRandom*. Generated password stored as **char[]**. Returns `Optional<char[]>`.

Why: *java.util.Random* uses a 48-bit Linear Congruential Generator (LCG). An attacker who observes even a few generated passwords can mathematically recover the internal seed and predict every future and past generated value. This completely breaks password generation security. *SecureRandom* uses the operating system entropy source (`/dev/urandom` on Linux, *CryptGenRandom* on Windows), which is cryptographically unpredictable. There is no safe fallback for cryptographic random number generation. The old silent fallback was silently catastrophic. Failing loudly (`RuntimeException`) is always preferable to silently using an insecure generator.

- **FileHelper.java — File Integrity Pre-Check**

Old Code: File passed directly to *EntriesRepository.readDocument()* with no prior validation

New Code: *verifyFileIntegrity(file)* checks: file exists and is readable; size > 0 and < 100MB; first 5 bytes match the JPass magic header ("jpass")

Why: Without a pre-check, a user could be tricked (via social engineering or a malicious download) into opening a crafted binary file, a zip bomb, or a file from an untrusted source. The crypto and parser layers would attempt to process it anyway, potentially exposing timing side-channels, triggering parser errors that leak internal details (FIX 16), or causing resource exhaustion. Verifying the magic header bytes before attempting decryption is standard defensive practice in file-processing security.

● **EntryHelper.java — Re-Authentication Before Destructive Operations**

Old Code: *deleteEntry()* and *duplicateEntry()* were gated only by a Yes/No confirmation dialog

New Code: *requireReAuthentication()* prompts for the master password and compares using *Arrays.equals(stored, entered)* on **char[]** before proceeding. The entered chars are zeroed immediately after comparison.

Why: Physical access to an unlocked machine combined with two mouse clicks was sufficient to delete all stored entries or duplicate and read any entry including its password, with no identity verification whatsoever. Identity verification before destructive or privileged read operations is standard practice in all serious password managers. The comparison uses *Arrays.equals()* on **char[]** rather than *String.equals()* to avoid creating interned String copies of the master password in memory.

● **FileHelper.java — XML XXE Prevention**

Old Code: XML files fed raw to the Jackson XmlMapper parser with no pre-flight validation

New Code: *validateXmlContent(file)* checks: file < 10MB; rejects files containing <!DOCTYPE, <!ENTITY, SYSTEM, or PUBLIC keywords (case-insensitive)

Why: An XML External Entity (XXE) attack works by embedding a DOCTYPE declaration in a malicious XML file that instructs the parser to read local system files and embed their contents in the parsed data structure. For example:

```
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<password>&xxe;</password>
```

Without this check the Jackson parser would silently read the contents of the referenced local file (/etc/passwd, /etc/shadow, or any other file the process can read) and embed those contents in the import. This pre-flight keyword check provides a defence-in-depth layer alongside disabling DTD processing in the XmlMapper configuration.

• **CryptOutputStream.java — SecureRandom for IV Generation**

Old Code: `Random rnd = CryptUtils.newRandomNumberGenerator()` — could silently return *java.util.Random*

New Code: `new SecureRandom().nextBytes(iv)` — direct, explicit, no wrapper, no possible fallback

Why: The AES-CBC Initialisation Vector (IV) must be cryptographically unpredictable. A predictable IV enables BEAST-style attacks (Browser Exploit Against SSL/TLS) where an attacker who can predict the next IV crafts a chosen plaintext whose first block XORs to a known value, then verifies guesses against the observed ciphertext. This turns AES-CBC encryption into effectively plaintext. The IV is written to the output file and is not secret, but it must never be guessable before the encryption happens.

• **AuditLogger.java — NEW CLASS — Security Event Logging**

What is new: Singleton class that appends structured one-line entries to `~/.jpass/audit.log`

Events logged include:

- Login attempts (success and failure) with file name
- Session lock and unlock events
- File open, save, export (flagged as unencrypted), and import
- Entry delete and duplicate operations (title only, never password)
- Clipboard copy of sensitive fields
- Master password change

Log injection protection: The `sanitize()` method strips newline characters and pipe characters from all field values, preventing an attacker from crafting a file name like `"x\nFAKE_EVENT"` that would corrupt the audit log structure.

Why: Without any audit logging there is zero ability to detect brute-force attempts, investigate a suspected data exfiltration after the fact, or know who accessed the application and when on a shared machine. Security audit logs are a fundamental requirement of any application that handles credentials.

● **DocumentProcessException.java — Safe Error Messages**

Old Code: `super("Cannot process document due to the following exception:\n" + message)` — raw OS/JVM message embedded and shown in the UI dialog

New Code: `safeMessage` field (generic, always shown in UI), `internalDetails` field (full technical text, for developer logs only). `getMessage()` overridden to return `safeMessage` so all existing callers automatically get the sanitized version.

Why: Raw JVM and operating system exception messages can expose sensitive internal details to any user who can trigger or observe the error dialog. Examples of what could be leaked:

- Filesystem paths: `/home/alice/.jpass/passwords.jpass` (Permission denied)
- Cipher identifiers: `AES/CBC/PKCS5Padding` — Bad padding exception
- Java class names: `com.fasterxml.jackson.databind.NullPointerException`

An attacker who can trigger or observe these dialogs learns internal architecture details — exact home directory structure, encryption algorithm identifiers, library versions — all of which are useful for crafting further targeted attacks against the system.

● **SearchPanel.java — Search Input Sanitization**

Old Code: `getSearchCriteria()` returned raw, completely unvalidated text from the `JTextField`

New Code: `sanitizeSearchInput()` trims whitespace, caps length at 256 characters, and strips all ASCII control characters (character codes below 32)

Why: Unbounded search strings with no length limit can cause performance degradation through excessive string comparison work across the entire entry list. ASCII control characters (codes 0–31)

including null bytes, carriage returns, and escape sequences have no legitimate use in an entry title search and could cause problems if the search term is ever used in a log file, a regex, or an XML context in future code changes. Sanitizing at the input boundary is a standard defence-in-depth measure.

- **Worker.java — Session Guard for Background Tasks**

Old Code: Background SwingWorker tasks executed unconditionally, regardless of session state

New Code: *isSessionActive()* helper method available — returns false if the session has expired. Guards against background tasks that were queued just before the inactivity lock fired.

Why: A race condition exists between the inactivity timer and any background operation that was queued or started just before the lock engaged. Without this guard, a save, open, or export worker could continue to run on in-memory credential data even after the user's session should be locked. The session guard ensures that background operations on sensitive model data are only permitted when the session is confirmed active.